

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE1110 — Análisis de Señales Mixtas

Taller semana 5

Experimentación de detección de ecos

Estudiante: Josue A. Pérez

Carné: 2025078641

Profesor: Luis Chavarría Zamora

Fecha de entrega: 16 de septiembre de 2026

Repositorio: https://github.com/JosueAPerez/jperez_asm_2026_s2

1. Introducción

Todo el taller fue realizado en Python utilizando las bibliotecas numpy y matplotlib, el código fuente además de estar presente en el archivo .zip, también se encuentra en el repositorio del proyecto al que corresponde este taller, siguiendo el mismo flujo de una rama master y una de development.

2. Parte 2: Experimentos con FFT

2.1 Implementación de la DFT y la FFT

Primero se implementó la Transformada Discreta de Fourier por fuerza bruta, lo cual consistió en calcularla en base a la definición matemática:

$$X[k] = \sum x[n] \cdot e^{(-j2\pi kn/N)} \quad k = 0, 1, \dots, N - 1$$

Esta implementación recorre para cada frecuencia k , todas las muestras n de la señal de entrada, por lo que termina teniendo una complejidad $O(N^2)$.

Segundo, se realizó la implementación de la Transformada Rápida de Fourier, esto siguiendo el algoritmo recursivo de Cooley-Tukey (radix-2). Este algoritmo lo que hace es dividir la señal de entrada en sus muestras de índice par e índice impar, calcula de forma recursiva la DFT de cada una de las mitades y combina ambos resultados mediante los “factores de giro”: $e^{(-j2\pi k/N)}$, al aplicar esto de manera recursiva hasta llegar a subproblemas de tamaño 1, el algoritmo se reduce a una complejidad de $O(n \log n)$.

Estas dos implementaciones fueron validadas utilizando la función `numpy.fft.fft` como referencia, esto confirmo que si hay una coincidencia exacta con las implementaciones (dentro de los márgenes de error).

2.2 Comparación de tiempos de ejecución

Para comparar el desempeño de ambos algoritmos se midió el tiempo de ejecución de la DFT, la FFT y de `numpy.fft.fft` para señales aleatorias de tamaños $N = 64, 128, 256, 512, 1024, 2048, 4096$, cada medición se repitió tres veces, reportándose el valor mínimo obtenido.

Tabla 1. Tiempos de ejecución para N =64 a 2048

N	DFT (s)	FFT (s)	Numpy.fft.fft (s)
64	0.004777	0.000144	0.00000550
128	0.019774	0.000292	0.00000520
256	0.074981	0.000633	0.00000830
512	0.312102	0.001469	0.00001460
1024	1.236574	0.002812	0.00001240
2048	4.788110	0.006076	0.00001910

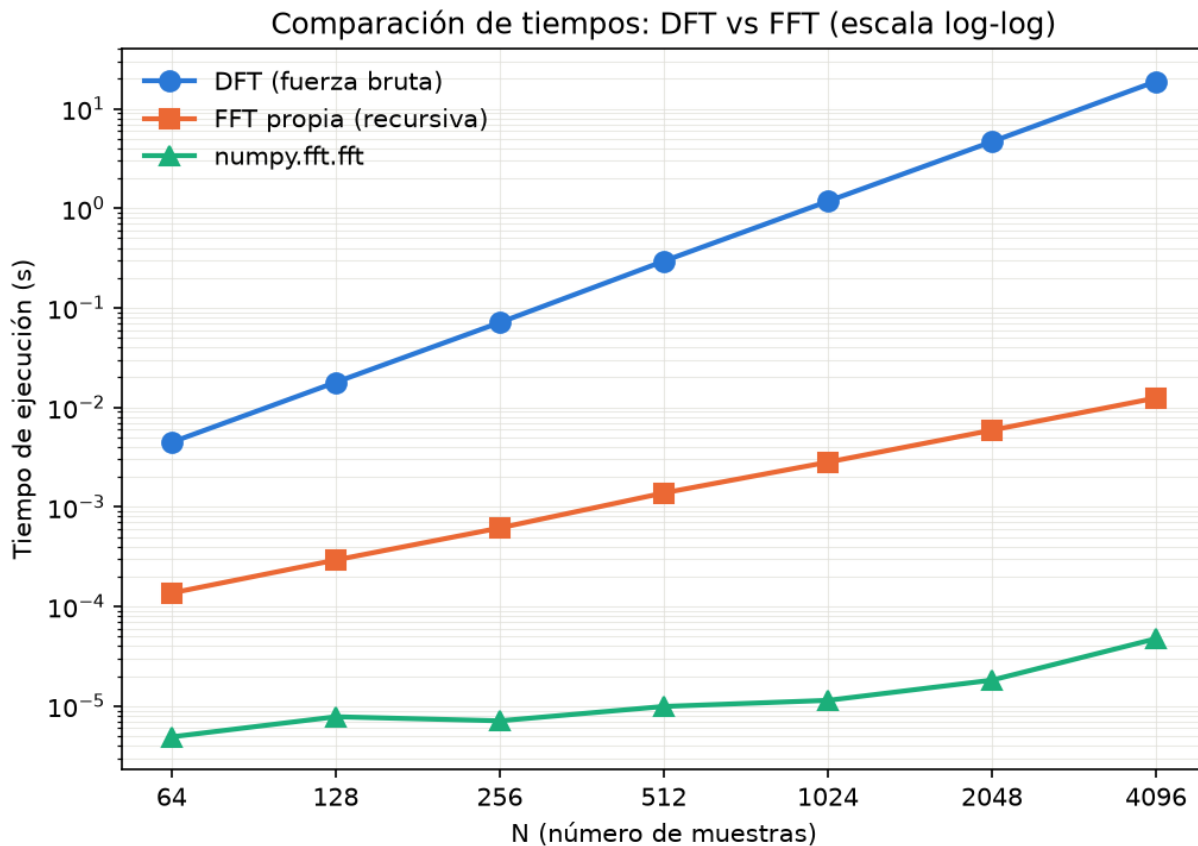


Figura 1. Comparación de tiempos de ejecución (escala log-log) entre la DFT, la FFT y numpy.fft.fft extendida hasta N = 4096

Los resultados confirman el comportamiento esperado de cada algoritmo, cada vez que N se duplica el tiempo de la DFT se multiplica por aproximadamente 4, el tiempo de la FFT se multiplica aproximadamente por 2, por la parte de numpy.fft.fft, esta se mantiene en el orden de

los microsegundos en todo el rango, esto se debe a su implementación optimizada en C, por eso el crecimiento de esta función solo se vería apreciada en valores muy grandes de N.

2.3 Representación de magnitud y fase

Para estudiar el comportamiento de la DFT en el dominio de la frecuencia se generaron cuatro señales de prueba de 256 muestras, utilizando una frecuencia de muestreo de 256 Hz (de este modo cada bin de frecuencia corresponde a 1 Hz), y se calculo la magnitud y la fase de ese espectro:

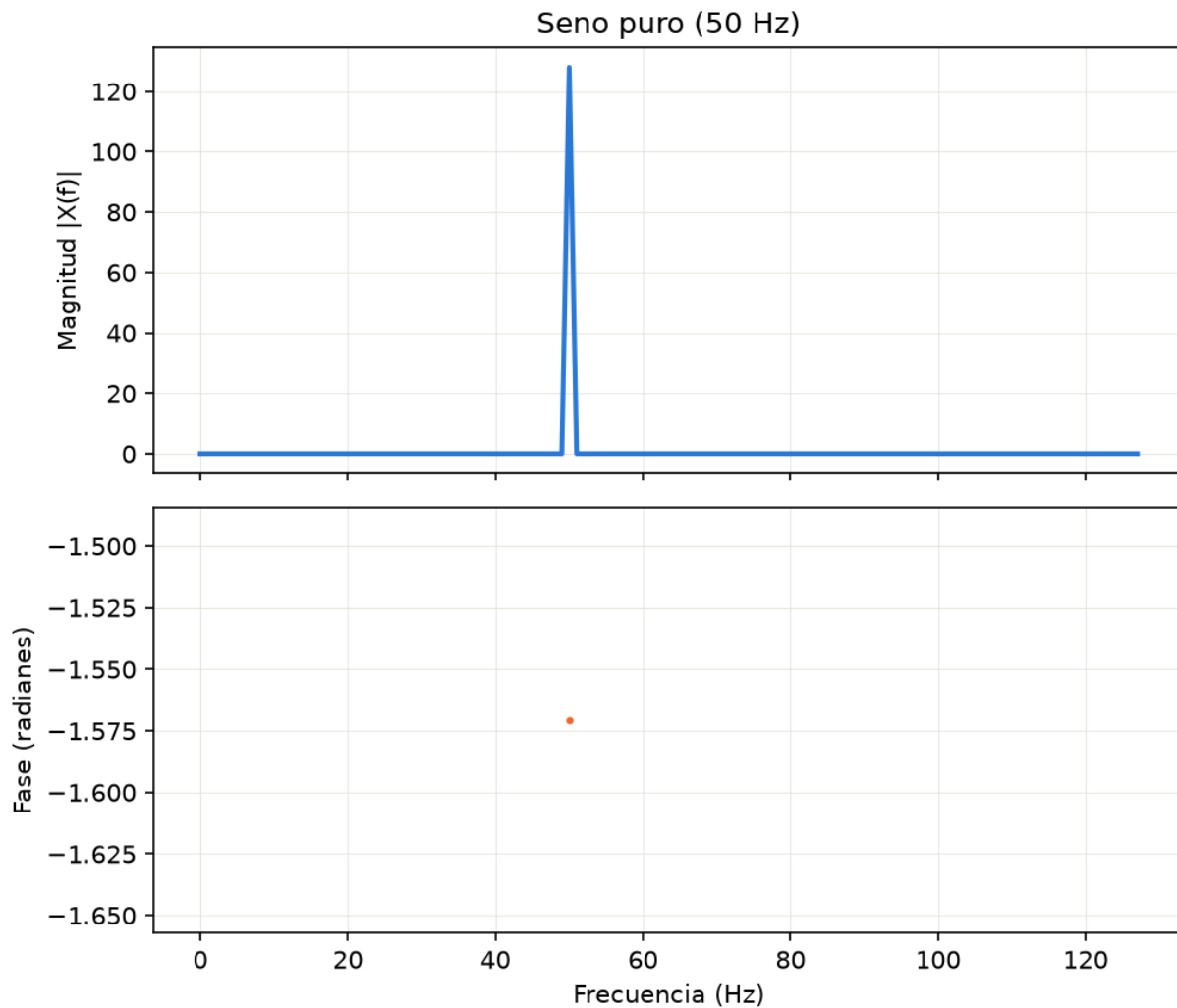


Figura 2. Magnitud y fase de un seno puro de 50 Hz

El espectro muestra un único pico limpio en 50 Hz con una magnitud de $N/2 = 128$, consistente con la amplitud unitaria del seno, la fase de este pico coincide exactamente con el desfase de $-\pi/2$ que sufre el coseno al representar un seno como un coseno.

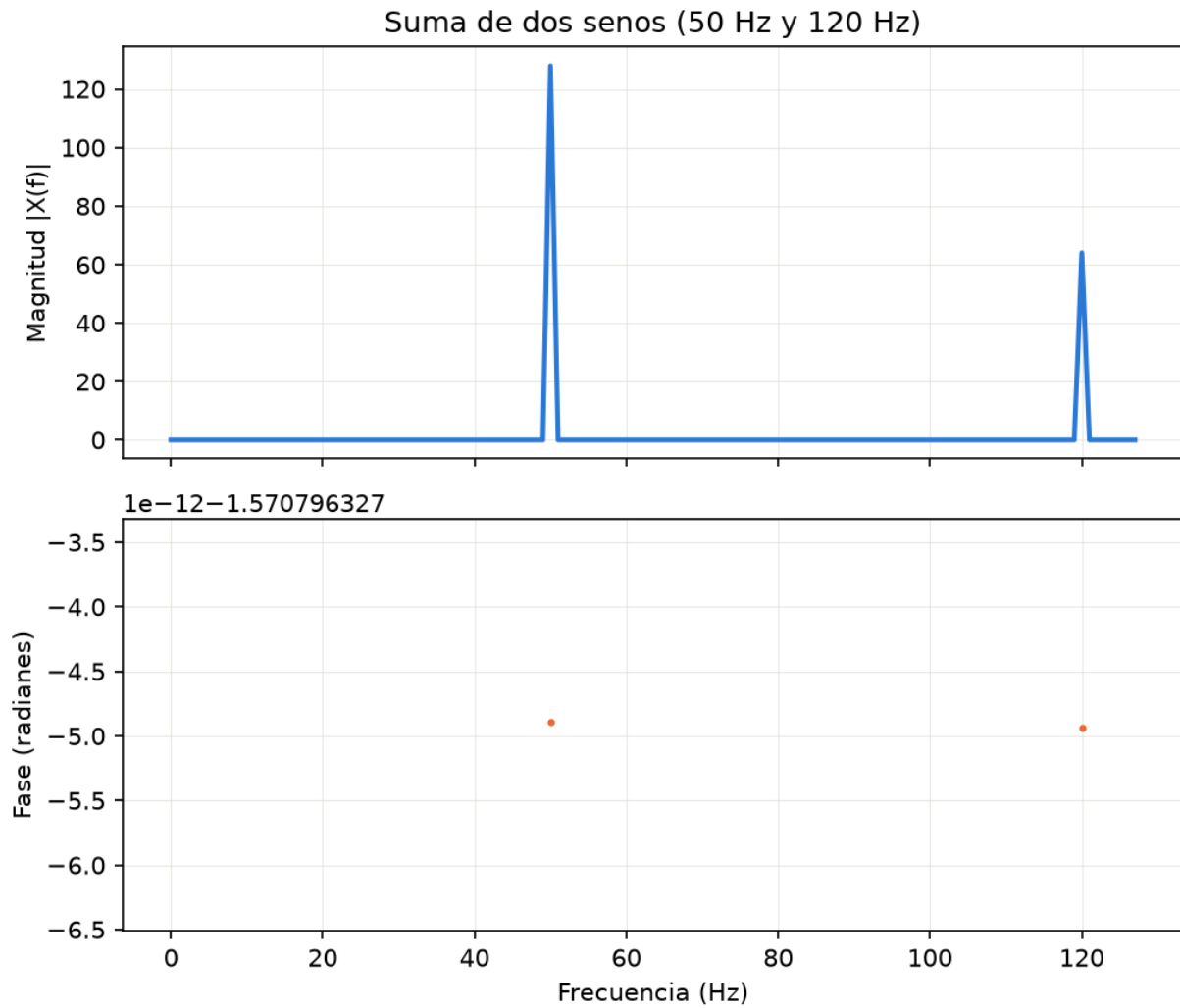


Figura 3. Magnitud y fase de la suma de dos senos (50 Hz y 120 Hz, con amplitudes 1 y 0.5)

Se pueden observar dos picos independientes, con magnitudes de 128 y 64, este resultado ilustra el principio de superposición, la transformada de una suma de señales es la suma de sus transformadas individuales.

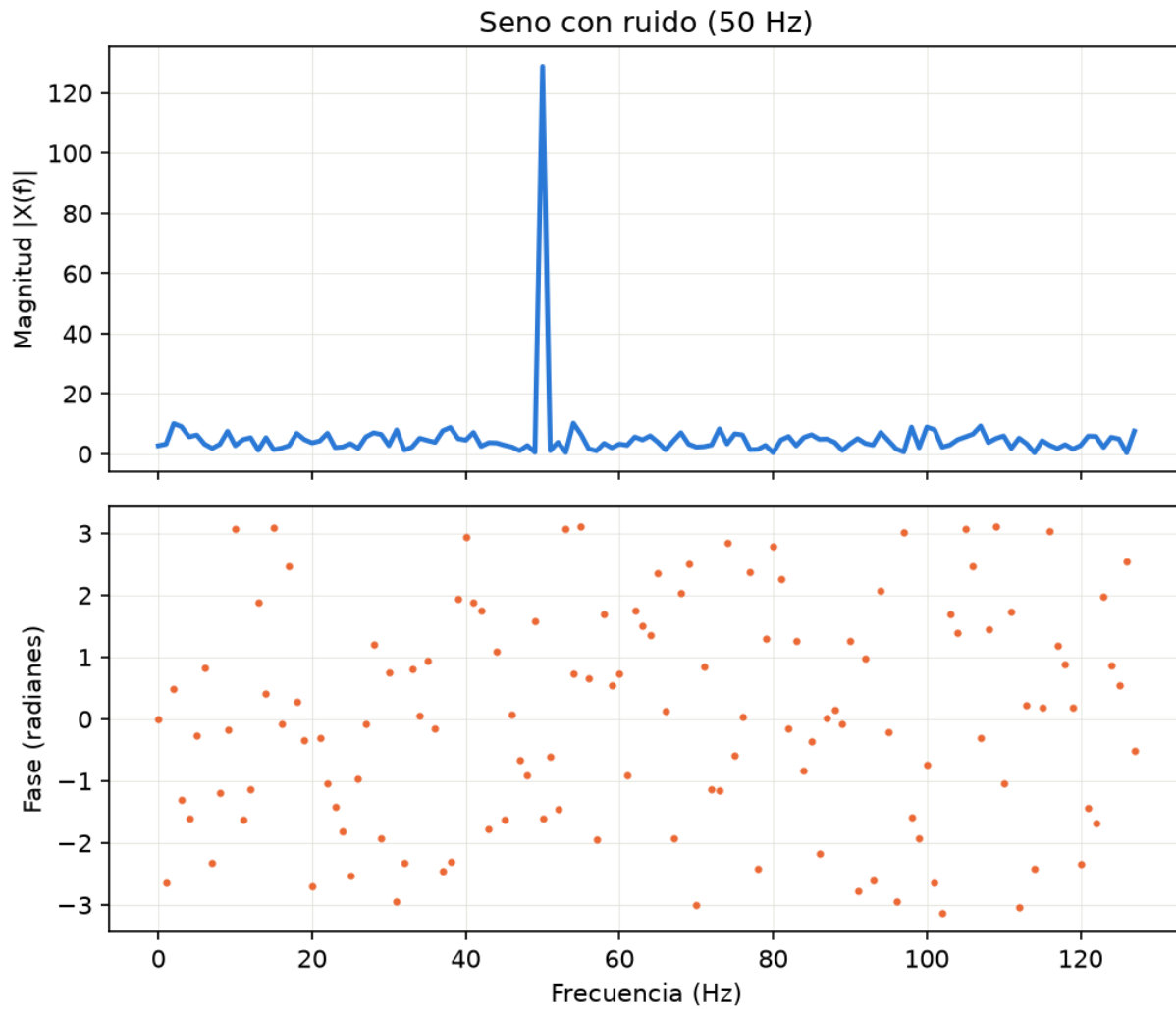


Figura 4. Magnitud y fase de un tono de 50Hz contaminado

A pesar del ruido, el pico que corresponde a la componente real de la señal permanece identificable por encima del ruido, la fase, por otro lado, resulta aleatoria en casi todos los bins, en esas frecuencias la magnitud no viene de una componente periódica real sino únicamente del ruido, por lo que la fase calculada no posee información relevante.

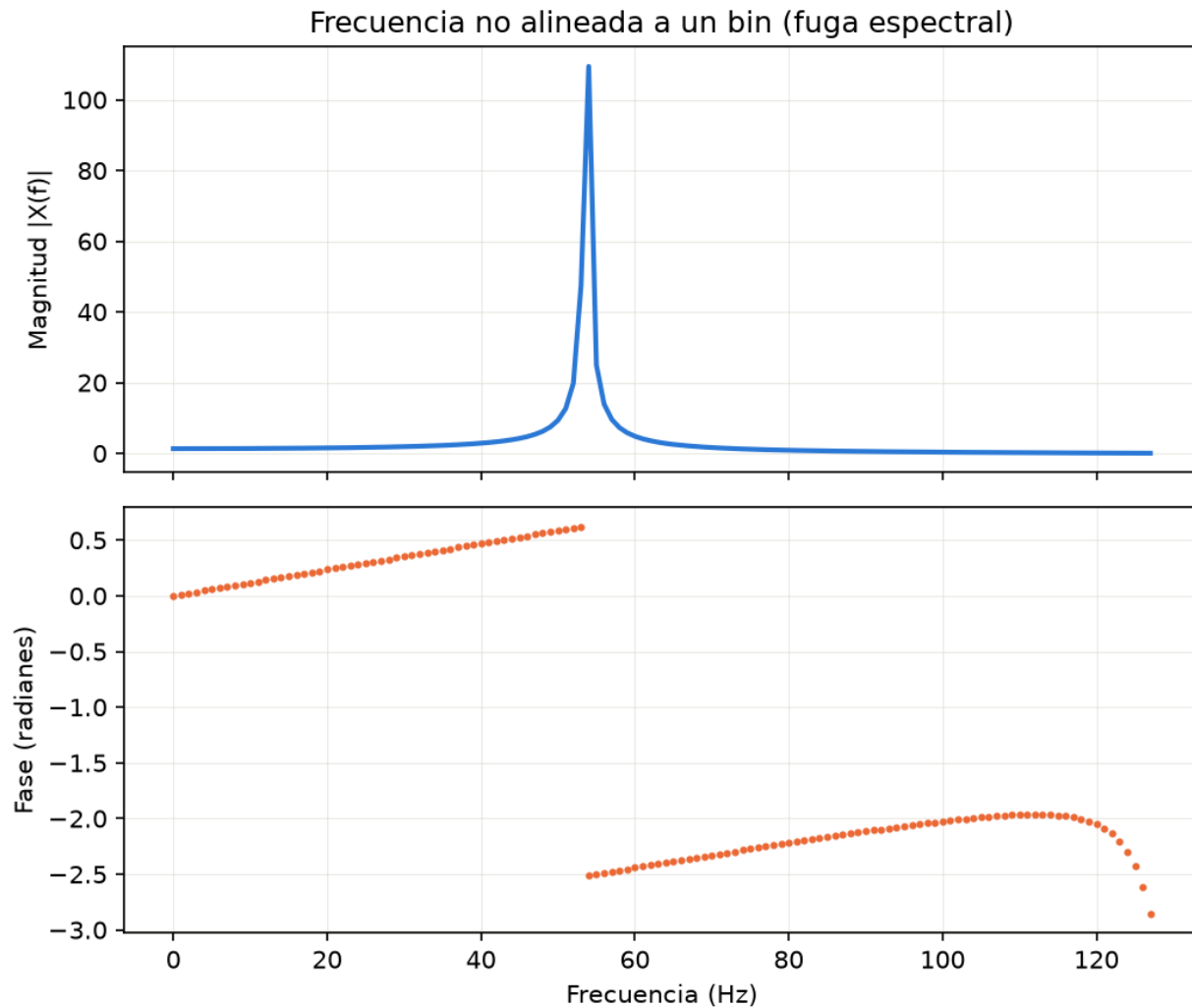


Figura 5. Magnitud y fase de una señal de 53.7 Hz, una frecuencia que no coincide con un bin

La señal al no coincidir con un múltiplo entero de la resolución de la frecuencia, la señal se dispersa entre varios bins vecinos en lugar de concentrarse en uno solo, este efecto es conocido como fuga espectral, esto ocurre porque toda señal de duración finita es de forma matemática, el multiplicar una señal infinita por una ventana rectangular, debido a que la multiplicación en el tiempo equivale a una convolución en frecuencia, el espectro ideal de la señal queda convolucionado con el espectro de la ventana esparciendo su energía.

3. Parte 3: Experimentos de detección de ecos

3.1 Generacion de la senial transmitida y su eco simulado

Para esta parte se genero un pulso conocido que consiste en un tono senoidal de 5ms de duración, a una frecuencia de 2000 Hz y una frecuencia de muestreo de 44100 Hz, a este pulso se le aplico una ventana de Hanning, la función de esta ventana es suavizar de manera gradual la amplitud del pulso desde cero hasta el valor máximo y de vuelta a cero, esto para que no inicie ni termine de forma abrupta, esto es importante por dos motivos, la primera es que un cambio abrupto de amplitud no es realista desde el punto de vista físico para un transductor real, y la segunda es que este mismo cambio abrupto introduce el mismo efecto de la fuga espectral, ósea, energía no deseada esparcida en frecuencias fuera de la banda de interés, la ventana lo que hace es concentrar la energía del pulso alrededor de su frecuencia real, produciendo un pulso más limpio.

Para simular la señal recibida del sistema se genero un buffer de 50 ms de duración y se inserto una copia atenuada y retrasada del pulso que se transmitió, representando el eco reflejado por un objeto, además se sumo ruido para simular condiciones de un micrófono real, el retardo se calculo a partir de una distancia hipotética de un metro, utilizando la relación:

$$d = \frac{v \cdot \tau}{2}$$

Con $v = 343$ m/s, obteniendo un retardo real de 5.831 ms (lo que equivale a 257 muestras a 44100 Hz), este valor calculado de antemano permitió verificar la precisión del algoritmo de detección.

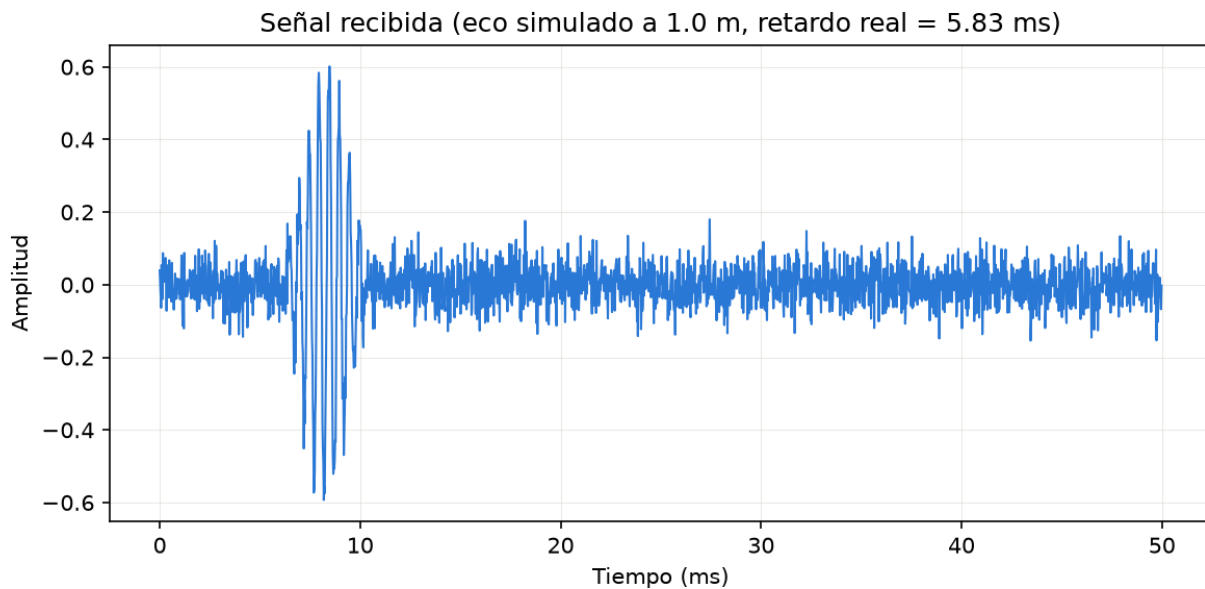


Figura 6. Señal generada con eco y ruido bajo

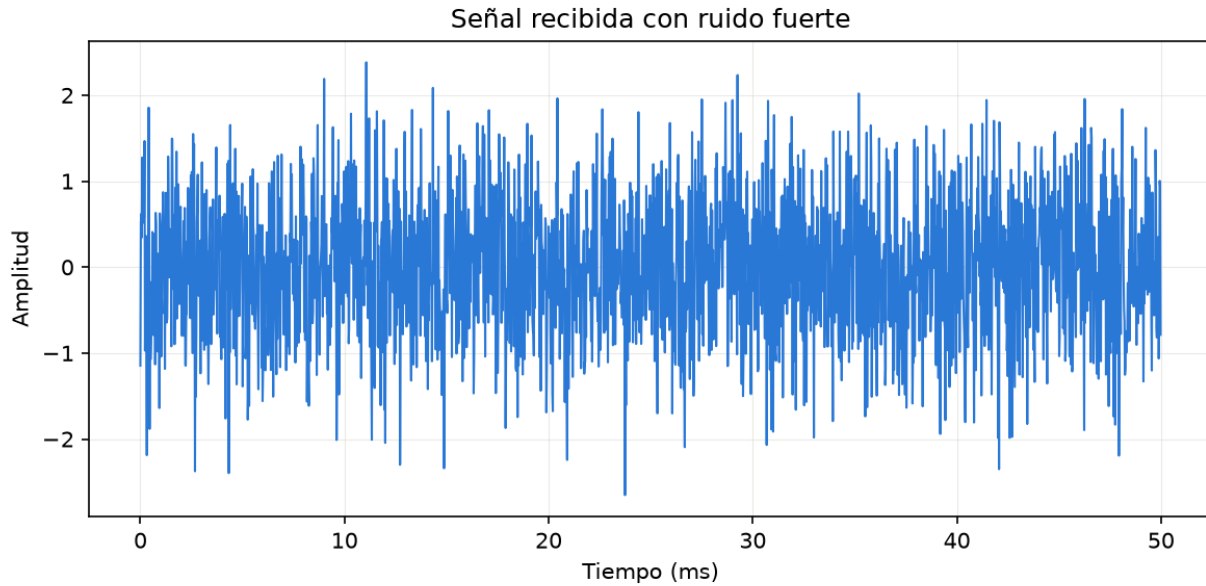


Figura 7. Señal generada con eco y ruido alto

3.2 Detección del retardo mediante correlación directa

Para estimar el retardo del eco dentro de la señal ruidosa se implementó una función de correlación que desliza el pulso conocido sobre la señal recibida, esto calcula en cada posición un producto punto entre ambos segmentos, en el lugar en el que el pulso coincide con el eco, en ese punto se va a generar un calor alto mientras que en el resto de posiciones los productos se cancelan de forma parcial entre sí, dando valores bajos, la posición del máximo de la curva del parecido corresponde a la muestra donde inicia el eco.

Al aplicar este algoritmo con la señal con ruido fuerte, el pico de correlación se ubico en la muestra 257, lo cual equivale a un retardo estimado de 5.828 ms, este valor coincide a nivel de muestra discreta con el valor de retardo real utilizado para generar el eco, la diferencia entre 5.828 ms y 5.831 ms corresponde al redondeo que se introduce al ubicar el eco en una muestra entera, ya que 5.831 ms no coincide exactamente con un múltiplo del periodo de muestreo, además, la función `numpy.correlate` dio un resultado similar.

Esto evidencia una limitación física, ya que la resolución de distancia del sistema esta determinada por la frecuencia de muestreo, ya que cada muestra representa un incremento temporal de $1/f_s$, en distancia se traduce a:

$$\Delta d = \frac{v}{2 \cdot f_s} = \frac{343}{2 \cdot 44100} \approx 3.9 \text{ mm}$$

Esto significa que el sistema no diferenciaría entre dos objetos que estén separados por esa distancia.

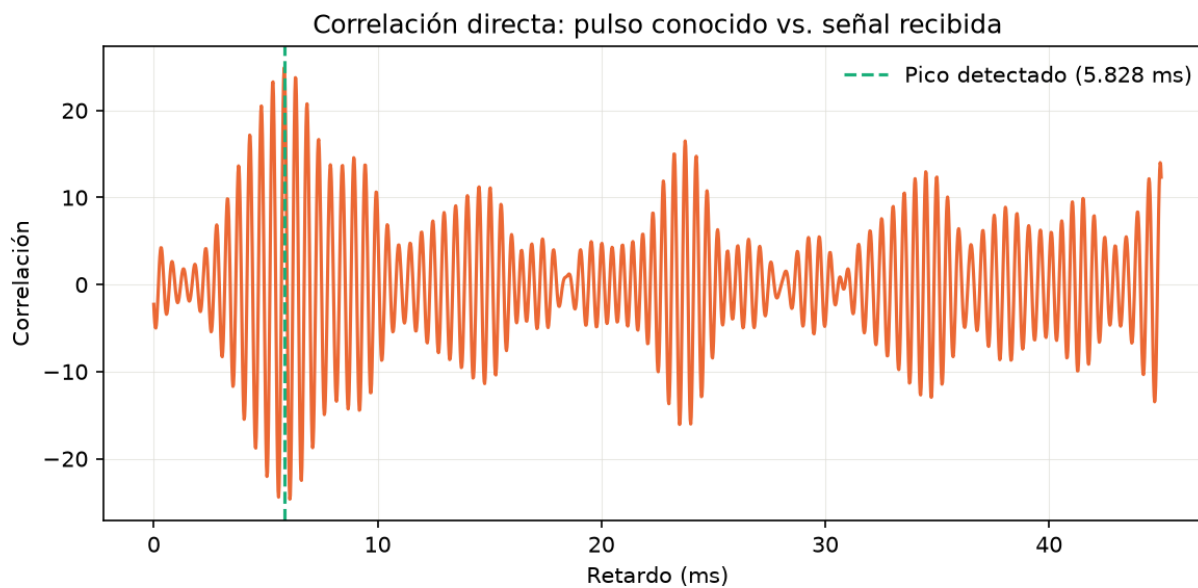


Figura 8. Curva de correlación con el pico marcado en el retardo detectado

3.3 Correlación mediante FFT

Utilizando la relación entre convolución y multiplicación en frecuencia, se realizó la implementación de una segunda versión del algoritmo, pero esta vez basado en FFT, dado que la correlación es equivalente a la convolución de una señal con la versión invertida en el tiempo de la otra, el algoritmo invierte el pulso conocido, calcula la FFT de ambas señales, multiplica ambos espectro y aplica la FFT inversa al resultado, además, se validó que esta implementación produjera resultados idénticos.

3.4 Comparación entre implementación directa y la basada en FFT

Además, se midió el tiempo de ejecución de ambas implementaciones sobre la misma señal, donde se obtuvieron los siguientes resultados:

Tabla 2. Comparación de tiempos entre correlación directa y basada en FFT

Método	Tiempo (s)
Correlación directa	0.001451
Correlación vía FFT	0.000188

La versión que se basaba en FFT resulto ser aproximadamente 7.7 veces más rápida que la implementación directa, esta diferencia es muy ventajosa para procesar señales de un tamaño mucho más considerable.

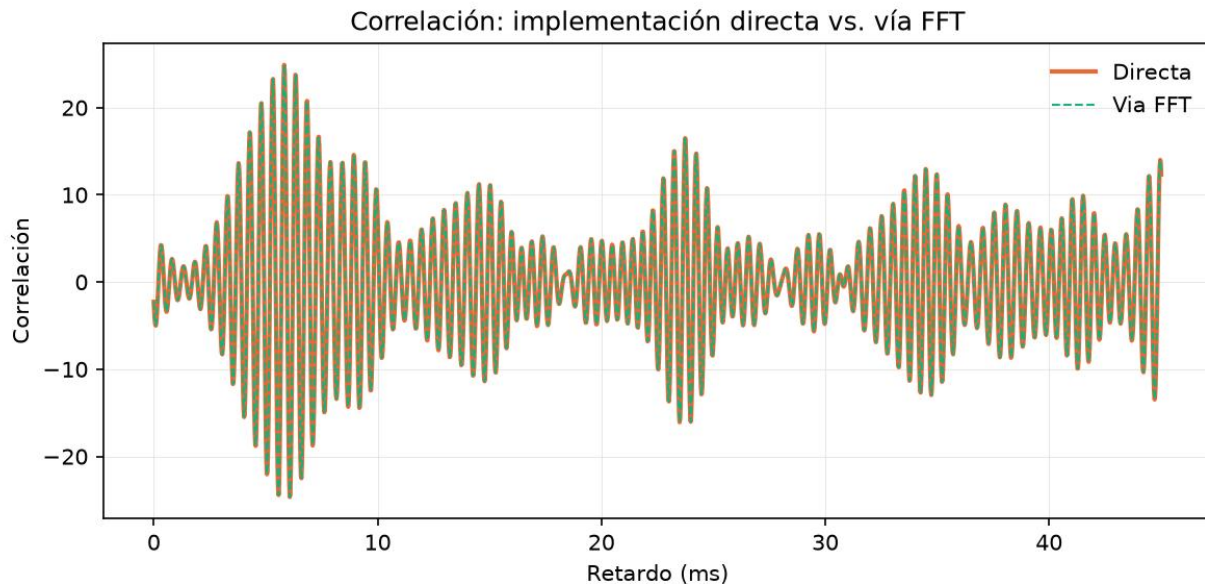


Figura 9. Comparación de ambas curvas de correlación

4. Conclusiones

- Se implementaron y verificaron de forma exitosa la implementación de la DFT y la FFT recursiva (radix-2), verificando su equivalencia numérica con `numpy.fft.fft`.
- El estudio de la magnitud y fase de distintas señales permitió observar efectos importantes como la superposición de componentes, la degradación del espectro por ruido y fuga espectral.
- Se logró generar una señal simulada con un retardo conocido y se demostró que la correlación es capaz de recuperar ese retardo.
- Se identificó la resolución de distancia del sistema y que esta se ve limitada por la frecuencia de muestreo empleada.

5. Recomendaciones

Se recomienda evaluar el desempeño de la correlación bajo distintos niveles de atenuación y ruido, con el fin de caracterizar el límite de detección del sistema antes de migrar la implementación al microcontrolador. Asimismo, dado que la implementación en microcontrolador no dispondrá de NumPy completo, se recomienda investigar con anticipación la biblioteca `ulab` para MicroPython, que ofrece una interfaz similar a NumPy, adecuada para hardware con recursos limitados.

6. Referencias

[1] A. V. Oppenheim, A. S. Willsky y S. H. Nawab, Señales y Sistemas, 2.^a ed. Ciudad de México, México: Prentice Hall, 1998.

[2] S. W. Smith, The Scientist and Engineer's Guide to Digital Signal Processing. San Diego, CA: California Technical Publishing, 1997. Disponible en: <https://www.dspguide.com>